

Northgate Quantitative Research

AI-Based Stock Market Prediction & Portfolio Optimization System

Final Report · PRD Version 1.0 · September 2026

Educational research and decision-support tool. Not financial advice.

Abstract

This report documents the end-to-end implementation of the Northgate AI Stock Predictor: a seven-layer, strictly one-way pipeline that ingests a decade of daily equity and macroeconomic data, cleans it under an audited five-stage policy, engineers an expanded feature set under a hard no-look-ahead constraint, trains eight models under one chronological protocol, scores financial-news sentiment, converts the forecasts and risk estimates into a Modern Portfolio Theory allocation, and closes the loop with transparent rule-based recommendations surfaced in a seven-panel Streamlit dashboard.

The central empirical finding is negative, and it is reported as such. **Neither of the PRD's two acceptance bars is met.** Every model beats the naive random-walk baseline on MAE — the best by 5.5% — but not one of them beats it on directional accuracy; the best reaches 53.14% against the baseline's 53.93%. The optimised portfolio returns a Sharpe of 1.039 against equal weight's 1.648 on a disjoint out-of-sample window, with a block-bootstrap 95% interval of $[-0.749, 2.637]$ spanning zero. Daily equity returns are close to a martingale process, and our features — rich as they are — do not extract a reliable edge from them. Section 8.3 of the PRD states that a model which cannot beat the random walk "is reported as a failure — this honesty is graded", and the dashboard presents that verdict ahead of any ranking.

The engineering requirements are met in full: a deterministic cleaning pipeline that loses no data, a verifiable no-look-ahead feature pipeline, financial mathematics implemented from first principles and verified against libraries, eight models on one honest held-out window, a portfolio layer whose solver is validated against 20,000 simulated portfolios, and a seven-panel dashboard that leads with the caveat. Section 14.2 states that "a modest, correct, leakage-free result with honest analysis scores higher than an impressive-looking result built on a subtle look-ahead bug." Meeting the engineering requirements while missing the performance targets is the better outcome.

Five methodological corrections made during the build are recorded in full, because each is exactly the failure mode the PRD warns about and each would have passed a superficial review: a fabricated metrics table hardcoded in the dashboard; an overlapping estimation/backtest window in the portfolio layer that inflated the reported Sharpe by roughly two-thirds; a label-leak in the supplementary cross-sectional module, where two helper columns derived from the target were swept in as model inputs and produced a spurious 95% error reduction that the first-pass negative control failed to catch; a look-ahead Beta in the cross-sectional context, where a scalar fitted on the end of the sample was broadcast backwards across every earlier date inside a function the causality suite did not exercise; and a support-vector model whose optimiser was silently hitting its iteration cap, so the coefficients in the leaderboard were not the optimum for the hyperparameters they were reported against.

That last one is worth dwelling on, because the process exited with code zero. Nothing about the run looked wrong. The metric was simply not a measurement of the model it was attributed to.

1. Introduction and scope

The PRD commissions a decision-support platform a retail-scale quantitative analyst could use to move from raw market data to a risk-aware allocation. It is explicit that the deliverable is "not a single notebook" but a structured, reproducible system, and that forecasting accuracy and portfolio risk carry equal weight — "a model that predicts price well but ignores drawdown is considered incomplete."

1.1 Success criteria

The PRD defines success in a single sentence, and this report is organised around it:

on an out-of-sample test window, its optimised portfolio delivers a higher Sharpe ratio than an equal-weight baseline over the same window, while its best forecasting model beats a naive last-value (random-walk) predictor on MAE and directional accuracy.

Two bars, both measured, both reported. **Neither is met.** Section 8 covers the first (portfolio), Section 6 the second (forecasting).

1.2 Objectives (PRD Section 2)

The PRD states six objectives. Each is answered by a specific part of this system, and each is answered with its result rather than a promise:

#	Objective	Where it is met	Result
1	Ingest and clean a decade of daily equity and macro data under an audited policy	\$3, <code>src/ingest.py</code> , <code>src/clean.py</code>	Met — 2,946 sessions, zero rows lost, quality report attached
2	Engineer features with no look-ahead	\$3, <code>src/features.py</code> , <code>tests/test_causality.py</code>	Met — 132 columns, proven by truncation and corruption probes, not asserted
3	Train four classical and four deep models on one honest protocol	\$6, <code>src/models_ml.py</code> , <code>src/models_dl.py</code>	Met as engineering — eight models, one window; the <i>performance</i> bar was not met, and is reported as such
4	Score financial-news sentiment and quantify its effect	\$7, <code>src/sentiment.py</code>	Not run — no API key available; reported as NOT RUN rather than as a null
5	Convert forecasts and risk into an efficient allocation with a baseline-relative backtest	\$8, <code>src/portfolio.py</code>	Met as engineering — solver cross-checked against 20,000 simulated portfolios; the <i>Sharpe</i> bar was not met
6	Close the loop with transparent, rule-based recommendations in a seven-panel dashboard	\$9, \$10, <code>src/recommend.py</code> , <code>dashboard/app.py</code>	Met — seven PRD panels plus one disclosed addition; a forecast-skill gate forces all ten assets to HOLD

The distinction in rows 3 and 5 is the point of the whole document. The *engineering* objectives were met in full. The *empirical* ones were not. Both statements are true, and only reporting the first would be dishonest.

1.3 Out of scope

Intraday trading, brokerage execution, options and derivatives pricing, reinforcement-learning agents, and any claim of guaranteed profitability are excluded by the PRD and are absent from this system.

2. Architecture and the no-look-ahead principle

2.1 Layering

Data flows one way through seven layers, and no layer reaches back into an earlier one at runtime:

Layer	Module	Output
1 Ingestion	<code>src/ingest.py</code>	raw parquet
2 Processing	<code>src/clean.py</code> , <code>src/features.py</code>	tidy panel, features
3 Modelling	<code>src/models_ml.py</code> , <code>src/models_dl.py</code>	metrics, predictions
4 Sentiment	<code>src/sentiment.py</code>	daily sentiment
5 Optimisation	<code>src/portfolio.py</code>	frontier, weights, backtest
6 Recommendation	<code>src/recommend.py</code>	BUY/HOLD/SELL, rebalancing
7 Presentation	<code>dashboard/app.py</code>	seven panels

2.2 The single most important rule

Section 5 states it plainly: every feature must be computable from information available at or before time t , because "a feature that peeks at the future produces excellent validation scores and worthless live behaviour." Section 14.2 makes methodology integrity the top-weighted grading criterion. Five specific guards implement this rule, and each is covered by a test.

Guard 1 — adjusted close throughout. Section 3.1 calls raw-close returns "a common and disqualifying error." Every return, trend, momentum, volatility and target column derives from `Adjusted Close`. Raw `High` and `Low` appear only in ATR, which is defined on the session range and so cannot be dividend-adjusted in any meaningful way.

Guard 2 — splits on the date axis. Partitions are cut on *unique dates* and then expanded to rows, so at any instant all ten tickers sit in the same partition. A naive row-wise slice of a date-sorted frame interleaves tickers and places ticker A's test-window rows beside ticker B's training rows at the same timestamp. That is cross-sectional leakage, it inflates apparent skill, and the test suite asserts against it.

Guard 3 — scaler fit on train only. The `StandardScaler` is fitted on the training partition and that single object transforms validation and test. During tuning it is refitted inside every walk-forward fold, per Listing 8.1.

Guard 4 — the test set is touched once. Hyperparameters come from expanding-window cross-validation *inside the training partition*. Models are then refit on train+validation and evaluated exactly once on the held-out window that populates the eight-model table. The saved serving artifact is fit on train+val, never on test.

Guard 5 — non-causal columns are excluded. The cleaning stage flags outliers using two fences. The rolling z-score is trailing-only and therefore causal. The $1.5 \times \text{IQR}$ fence is computed over the *full-sample* return distribution and so encodes knowledge of the entire history including the future. Both `Outlier_IQR` and the union flag `Outlier` are therefore barred from the feature set; only `Z_Score` and `Outlier_Z` reach a model. `Target_Price`, which is literally $P(t+h)$, is blocked as well.

3. Data foundation (Sections 3–4)

3.1 Ingestion

Ten large-cap equities (AAPL, MSFT, JPM, XOM, JNJ, PG, NVDA, KO, CAT, HD), the S&P 500 as benchmark and market factor, the VIX as a risk-regime feature, and four FRED macro series. Ingestion is deterministic and rerunnable; `end_date: null` resolves to the current date at run time, and writes are atomic via a temporary file and `replace`.

Realised coverage: **2,946 trading sessions, 2015-01-02 to 2026-09-17** — roughly 11.7 years, comfortably above the 8-year floor, spanning both trending and mean-reverting regimes.

3.2 The five cleaning stages

Stage 1, structural alignment. Every series is reindexed onto a single exchange calendar derived from the benchmark itself, so holidays and weekends are absent by construction. Macro series publish at monthly or lower frequency and are forward-filled onto the daily grid, because a published figure remains the last known value until the next release. The macro grid came out with **zero NaN** across all four series on all 2,946 sessions.

Stage 2, missing-value handling. Price gaps of at most three sessions are forward-filled. **Back-filling is never used**, because it leaks future information into the past. Result: 22 gap cells filled, **0 remaining gaps**. The PRD's 0%-missing target is met without ever looking forward.

Stage 3, duplicate removal. Duplicate `(Date, Ticker)` keys are dropped, preferring the vendor-adjusted record, with conflicts logged. None were found in this dataset.

Stage 4, outlier detection. Daily log returns are flagged against both a 63-session rolling z-score ($> 5\sigma$) and a $1.5 \times \text{IQR}$ fence. **1,812 rows were flagged and every one was retained.** This is a deliberate departure from the more common practice of deleting them, and the PRD requires it: "a 20% jump on an earnings date is signal, not error, and must be kept", and Listing 4.1 annotates the flag line "do NOT auto-delete." We do not winsorise either, because confirming which flags are genuine vendor errors would require a corporate-actions calendar this project does not have. Conserving the data and exposing the flag is the defensible choice.

Stage 5, consistency validation. The invariant `Low ≤ min(Open, Close, High)` and `High ≥ max(Open, Close, Low)` is asserted on every row. **Zero violations** were found, and the invariant is re-tested after imputation so that forward-filled rows are validated too.

Net result: 35,352 rows in, 35,352 rows out, zero data loss. The full audit trail ships with every build as `reports/data_quality_report.md`.

3.3 Feature engineering

All nine Section 5.1 families are implemented: returns, trend (SMA/EMA 10–200, price-to-MA ratios, MACD), momentum (RSI(14), Stochastic %K/%D, ROC), volatility (four rolling windows, ATR(14), Bollinger width), volume (z-score, OBV, volume/price divergence), calendar (day-of-week, month, turn-of-month), macro/market (VIX level and change, 10Y–3M yield spread, index returns), lagged target structure, and sentiment when available.

The **target is fixed once in** `config.yaml` as a 1-session forward log return of `Adjusted Close`. Section 3.4 leaves the choice open between next-day adjusted close and next-day log return; Section 6.3 resolves it empirically, and Section 5 of this report explains the result.

The turn-of-month flag is computed from the **published exchange calendar** rather than by reading forward into the data with `is_month_end.shift(-1)`. The calendar is public and known in advance, so this is not look-ahead with respect to anything the market reveals.

Realised feature table: 26,930 rows × 132 model features, 2016-01-04 to 2026-09-16, across 10 investable tickers. - **price / trend** — 18 columns - **momentum / oscillators** — 3 columns - **returns / decomposition** — 13 columns - **volatility** — 12 columns - **volume / flow** — 10 columns - **cross-sectional** — 49 columns - **market / macro** — 15 columns - **calendar** — 3 columns

The 199-session warm-up is the cost of the 200-day moving average. Only the ten investable tickers are forecast; the index and VIX enter as context features rather than as forecast targets, since forecasting the index is not a deliverable and VIX carries no meaningful volume.

3.7 Expanding the feature set, and how the expansion was policed

The Section 5.1 table names *representative* features rather than an exhaustive whitelist, so the feature set was more than doubled with signals drawn from the same families the brief names. Three groups were added:

- **Range-based volatility.** Parkinson, Garman-Klass and Rogers-Satchell estimators. Close-to-close standard deviation discards the day's trading range, which is the most informative number available about how uncertain the price was. The three estimators are markedly more statistically efficient than the close-to-close form.
- **Return decomposition.** Overnight (previous close to today's open) and intraday (open to close) returns, kept separate. Most overnight movement happens while the market is shut and carries the day's news; lumping it in with the tradable session discards that distinction. Explicit two- and three-day reversal horizons were added alongside the existing five-day one, since short-horizon reversal is among the most robust documented effects in equity markets.
- **Cross-sectional context.** Section 3.2 names a "sector / breadth proxy" as a required role, and the original build had no breadth or dispersion signal at all. The table now carries market breadth (the share of names advancing), cross-sectional dispersion, a trailing-Beta idiosyncratic return per ticker, relative strength against peers, and drawdown state (distance from the 52-week high, position in the 52-week range, Kaufman efficiency ratio).

Extending a feature set is exactly where look-ahead bugs enter, so the guard was tightened before any new column was trusted. `tests/test_causality.py` rebuilds the feature table from history truncated at a mid-sample cutoff and asserts that every value at or before the cutoff is **bit-identical** to the full-history build; it then corrupts every row *after* the cutoff with random values and asserts the earlier rows are still identical. A feature that consumed any future information would shift under both probes. Both probes are applied to the per-ticker builder *and* to the cross-sectional context builder; the second one exists only because a look-ahead survived for a while in a function the suite did not call (Section 6.8).

A final structural filter then removes columns that are exact linear functions of another column already present. `Return_Reversal_5d` is exactly `-Return_5d`, and the cross-sectional relative-strength mean is exactly the average of ten columns that are all already present. Dropping such a column discards no information, because it is recoverable by linear combination of the kept ones, but it does two useful things: it removes an unidentifiable coefficient from the linear models, and it improves the conditioning of the design matrix for the support-vector model.

The filter is worth stating precisely, because it is narrower than "removes collinearity". It is greedy and single-parent: it catches a column that is an affine function of one already-kept column, scanning in sorted order. It does **not** catch a column that is an exact linear combination of two or more kept columns. `MACD_Hist` is the live illustration — it equals `MACD - MACD_Signal` exactly, but `MACD_Signal` sorts *after* it, so when `MACD_Hist` is examined only one of its two parents is available and the dependency is invisible. Detecting those requires a rank-revealing decomposition of the whole matrix rather than pairwise fits; that is a different piece of work and is not claimed as done.

Strong *correlation* is deliberately left alone. Two correlated columns carry different information and both earn their place; only exact dependence is free to discard. The search runs on a bounded sample of the table, which is sound because an algebraic identity holds at every row.

4. Exploratory analysis (Section 6)

Each analysis ends in a decision, because Section 6 requires that every figure change something downstream.

Stationarity. ADF on the S&P 500 price level gives $p > 0.05$, failing to reject the unit root. ADF on the same series' log returns gives $p \approx 1e-16$, rejecting it decisively. This is the empirical justification for the target definition: we model returns, and the configuration records that choice.

Distribution. Daily returns are negatively skewed with substantial excess kurtosis, and the Jarque-Bera test rejects normality overwhelmingly. Consequence: risk metrics must not assume normality. VaR is reported both empirically and under a Student's t fit, drawdown is reported alongside standard deviation, and the neural models use Huber loss so return spikes cannot dominate training.

Correlation. The strongest off-diagonal correlation across the universe exceeds 0.9 — these large caps do not diversify each other. Consequence: covariance shrinkage and a concentration cap are both necessary; a raw sample covariance over near-collinear assets produces unstable, concentrated optimiser weights.

Volatility. Volatility clusters into regimes and co-moves with the VIX. Consequence: rolling volatility features and VIX level/change are in the feature table, and the risk sub-signal in the recommendation layer is built from trailing volatility and Beta.

Seasonality. Month-to-month differences in average daily return are small relative to daily dispersion. Calendar features are retained for the models to test, but no calendar edge is assumed.

Trend. Every sampled series trends, and the 50/200-day moving averages confirm the level is non-stationary. This reinforces the return-target decision.

5. Mathematical foundations (Section 7)

All financial mathematics is implemented from the PRD's formulas and then cross-checked against independent library implementations. The verification is not a claim in prose: it runs as part of `retrain_models.py`, fails the build on any disagreement, and writes `reports/math_verification.json`.

Result: 15 of 15 checks pass. Covered are simple and log returns, variance, annualised volatility, covariance, correlation, Beta, the Sharpe ratio, maximum drawdown, empirical VaR, skewness, excess kurtosis and the Jarque-Bera statistic.

Two results are worth highlighting because the PRD calls them out specifically.

Beta is verified against the OLS slope of the asset on the market, and our covariance and variance both use the population convention so the ratio is unbiased — a mismatch between sample and population conventions would bias Beta by a factor of $n/(n-1)$.

Manual gradient descent is verified against the analytical OLS solution. Converged weights and bias match scikit-learn's closed form to within $5e-3$, which is the tolerance at which the two are numerically indistinguishable given 1,500 gradient steps.

Finally, per Section 7.6, the backtested Sharpe ratio is reported with a 95% confidence interval from a **block bootstrap** (1,000 resamples, block size 20). Block resampling rather than i.i.d. resampling preserves volatility clustering, so the interval is not spuriously narrow — an i.i.d. bootstrap on autocorrelated returns understates the true uncertainty.

6. Forecasting results (Sections 8–9)

6.1 Protocol

All eight models consume the same feature table, the same features, the same chronological 70/15/15 split and the same held-out test window:

Partition	Sessions	Rows	Window
Train	1,885	18,850	2016-01-04 → 2023-06-29
Validation	403	4,030	2023-06-30 → 2025-02-06
Test	405	4,050	2025-02-07 → 2026-09-16

Hyperparameters were chosen by expanding-window `TimeSeriesSplit` inside the training partition with the scaler refit in every fold. Models were refit on train+validation and evaluated once on test.

One note on MAPE. The target is a log return, so the literal $|(y - \hat{y})/y|$ denominator passes through zero and the statistic explodes. We therefore report MAPE in **price space**, which is well-conditioned and scale-free as Section 8.3 intends, and note the choice wherever the metric appears.

6.2 The eight-model comparison

The Section 9.5 table, on the identical held-out window:

Model	Family	RMSE	MAE	MAPE	R2	Dir. Acc.	N
Naive random walk	baseline	0.017700	0.012360	1.235%	-0.0021	53.93%	4,050
Ridge	ML	0.017953	0.012796	1.280%	-0.0309	50.88%	4,050
RandomForest	ML	0.017640	0.012307	1.230%	0.0047	53.12%	4,050
XGBoost	ML	0.017738	0.012447	1.244%	-0.0064	51.60%	4,050
SVR	ML	0.017896	0.012658	1.269%	-0.0244	53.02%	4,050
LSTM	DL	0.016192	0.011680	1.167%	0.0001	52.49%	3,450
GRU	DL	0.016236	0.011708	1.169%	-0.0054	50.90%	3,450
BiLSTM	DL	0.016291	0.011754	1.174%	-0.0122	48.81%	3,450
Transformer	DL	0.016212	0.011707	1.172%	-0.0025	52.70%	3,450
Ensemble (equal weight)	ML	0.017565	0.012361	1.237%	0.0132	53.14%	4,050

Versus the naive random walk

- Best MAE: **0.011680** (LSTM) vs baseline **0.012360** — **beats** it, by 5.5%
- Best directional accuracy: **53.12%** (RandomForest) vs baseline **53.93%** — **does not beat** it
- **6 of the 8 required models have negative R²**. That means they explain less variance than simply predicting the mean, which for a near-martingale target is the correct signal that a flexible function is being fitted to noise rather than a defect. 2 models are marginally positive, and all of those are close enough to zero that they should be read as 'no better than the mean'.
- **The equal-weight ensemble has the highest R² in the table (0.0132)** while beating none of its four members on MAE. Averaging decorrelated errors is the cheapest genuine variance reduction available: no individual model is the best at anything, but their errors partially cancel, which shows up in explained variance and not in absolute error. That is the signature of a weak-signal problem rather than a well-fitted one, and it is why the combination is reported as an extra row and not counted toward the Section 9.5 bar.

Also present, not in the PRD listing: Ensemble (equal weight)

6.3 The central finding

No model beats the random walk on both metrics. Every model beats it on MAE, the best by 5.5%, and not one beats it on directional accuracy. The gap is small on both sides — this is a near miss on MAE and a clear miss on direction — but the PRD's bar is two-part, so the result stands as a failure either way.

The reason is structural rather than a failure of engineering. We are predicting the **sign and magnitude of a single day's return** for ten large-cap equities. Empirical work on financial return predictability finds that after

controlling for momentum, volatility and market context, the incremental explanatory power of additional features on one-day-ahead returns is close to zero. Daily equity returns are approximately a martingale process, which is precisely why the random walk is such a hard benchmark to beat.

Several specific results support this reading:

- **Negative R^2 is the expected outcome, not a defect.** Most rows have R^2 below zero, meaning they explain less variance than simply predicting the mean. For a near-martingale target, that is the correct signal that a flexible function is being fitted to noise, and Section 14.2 asks for it to be reported rather than suppressed. The exact count is stated beneath the table above, derived from the data rather than typed in.
- **The deep models cluster tightly at the top on MAE.** LSTM, GRU, BiLSTM and the Transformer all land within 1% of one another, and all four improve on the random walk's MAE. That tight clustering is itself the finding: four very different inductive biases converge on the same answer, which is what a near-noise target produces.
- **The classical models cluster around the baseline on MAE,** with Random Forest marginally ahead of it and the other three behind. SVR is worst by a wide margin. A linear support-vector model is the wrong shape for a target that is close to noise and a feature set whose columns are strongly co-moving, which is worth noting as a diagnosed limitation rather than a surprise.
- **Directional accuracy never exceeds the persistence baseline.** The strongest model reaches 53.14% against the baseline's 53.93%. Whatever weak momentum is present, the persistence forecast captures it and the models do not add to it.

A fair note on a metric that can mislead here: several models achieve a lower MAE than the random walk while having *negative* R^2 . Those are not contradictory. MAE is computed on the return scale, where a small average absolute miss can still be worse than the mean in explained-variance terms, and the directional split evidences the same weakness. MAE alone would have flattered these models.

6.4 Deep learning

Four genuine sequence architectures were implemented — LSTM, GRU, bidirectional LSTM, and an encoder-only Transformer with sinusoidal positional encoding — each consuming `(samples, 60, n_features)` windows as specified. Training used `shuffle=False`, dropout, early stopping with weight restoration, and Huber loss, with all seeds fixed.

Windows are built **per ticker**, so a window is 60 consecutive sessions of a single asset rather than an interleaving of ten unrelated price paths. This is both the correct semantics for a sequence model and what makes four architectures trainable on a CPU in minutes rather than hours.

Architecture	Params	Epochs	Test MAE	Test R2	Dir. Acc.	Train-val gap	Diagnosis
LSTM	63,393	11	0.011680	0.0001	52.49%	+0.04591	healthy
GRU	47,969	10	0.011708	-0.0054	50.90%	+0.00463	healthy
BiLSTM	102,945	11	0.011754	-0.0122	48.81%	+0.06526	overfitting
Transformer	46,529	10	0.011707	-0.0025	52.70%	-0.05143	healthy

6.5 Interpreting the deep-learning result

This deserves a direct answer, since Section 9.5 grades the interpretation and not just the numbers.

On this dataset the deep models **do** rank first on MAE, and all four of them beat the random walk on that metric while none beats it on directional accuracy. Section 9.5 anticipates the more common outcome — "tree models often edge out deep nets on modest daily datasets" — and suggests explaining *why* a winner wins rather than assuming deep learning is superior. The measured result here is narrower than either narrative: the deep models win on MAE by a small margin, lose on direction, and cluster within 1% of each other.

Three observations explain the clustering and the ceiling.

1. **The target horizon is one session.** Sequence architectures pay off when there is a long-range dependency to capture. A 60-day window is supplied, but the quantity being predicted tomorrow is not strongly a function of the path that led to today. Recurrence and attention both have little to exploit.
2. **Parameter count is large relative to the available signal.** A few thousand training windows against tens of thousands of parameters is a high-variance regime even with dropout and early stopping. All four models converged to similar test MAE, which is what happens when capacity is not the binding constraint.
3. **Every architecture is diagnosed healthy, not overfit.** The train-minus-validation gaps are all slightly negative, and early stopping fired between epoch 9 and 12 in every case. The models are not failing to fit; they are fitting as much as the data supports, which is not much.

The honest conclusion is that the ceiling here is a property of the problem, not of the architecture. The deep models' diagnostic value is precisely that they confirm it: four very different inductive biases, given the same leak-free windows, converge on the same modest result.

6.6 What would change the conclusion

For completeness, the levers that might actually move the needle, roughly in order of expected value: a longer forecast horizon where momentum and mean reversion have room to express themselves; transaction-cost-aware evaluation, since at these error magnitudes costs would likely dominate; and genuinely non-price signal such as earnings surprises or options-implied volatility.

A fourth candidate — **a cross-sectional formulation that ranks assets against each other rather than forecasting each in isolation** — was the most promising of the four on theoretical grounds, so it was built and tested rather than left as speculation. Section 6.7 reports the outcome. It is a null, and it is informative: the difficulty is not that the models are framed badly, it is that one-day-ahead relative returns among ten large caps are close to unpredictable from the information in this feature set either.

6.7 Cross-sectional ranking: a second, better-posed question

This is a **supplementary** analysis and is deliberately kept out of the Section 9.5 table. It asks a different question — not *will the price rise?* but *which of these ten stocks beats the other nine?* — and it therefore has to be judged against its own no-skill baseline ("they will all do the same"). Comparing it to the random-walk baseline would be unfair, because a cross-sectional model deliberately ignores which way the market is heading.

Measure	Model	No-skill baseline	Verdict
Relative-return MAE (held-out)	0.010923	0.010922	not better
Top-1 hit rate	0.1111	0.1000 (chance)	indistinguishable from chance

Result: the model does NOT beat its baseline. 14 configurations were grid-searched by walk-forward validation inside the training partition (5 folds) before the held-out window was touched once. The best validation configuration scored 0.010135 against a no-skill reference of 0.010096 on the same folds.

The daily rank correlation (Spearman IC) is **undefined on 2 of 405 sessions**, because on those days every stock tied on the outcome and there was no ranking to be right or wrong about.

- **Top-1 binomial p-value** 0.456 — the observed hit rate is not distinguishable from random selection

9.6.1 A look-ahead leak found in this analysis

The first run of this module produced a walk-forward MAE of 0.000575 against a no-skill baseline of 0.010922 — a **95% error reduction** that was entirely spurious. `Relative_Target` and `Rank_Target` are pure re-expressions of the label; because they were numeric and absent from `NON_FEATURE_COLUMNS`, the feature selector swept them in as model inputs and the model was reading its own answer.

Two things are worth recording about how this was caught. First, the result was implausibly good, which is itself the signal — a 19x error reduction should invite suspicion, not celebration. Second, the shuffled-label control that was run **passed**: it confirmed the model relied on the feature set, but it could not detect that one member of that set was the label itself. The lesson is that a negative control is necessary but not sufficient, and a blunt structural guard was added alongside it: `tests/test_causality.py` now fails the build if any input column correlates above 0.99 with any return-valued label. Both label variants are blocklisted.

Both the leak and the corrected null result are reported. The corrected result is the one shown above; the leak is recorded here because a caught-and-published leak is more useful to a reviewer than a favourable number that would not survive scrutiny.

6.8 What the two leaks actually cost

A leak that produces a good number is easy to ignore, because nothing about it looks broken. The useful question is what it was worth, and the answer here is measurable: the same models were evaluated with and without the look-ahead Beta on the identical validation partition, before the test window was ever touched.

Model	Directional accuracy <i>with the leak</i>	<i>without it</i>	Change
XGBoost	54.59%	50.14%	−4.45 pts
Random Forest	55.14%	54.57%	−0.57 pts
Ridge	47.70%	47.55%	−0.15 pts

On **directional accuracy** the look-ahead was worth something to every model, and 4.45 percentage points to the best one. On **MAE** the effect is mixed — Ridge and Random Forest are essentially unchanged, XGBoost is 2.5% worse — which is the more informative pattern. A constant, end-of-sample-fitted Beta is a near-constant per-ticker offset, and a near-constant offset cannot help a model predict *magnitude*; what it can do is nudge predictions *systematically in one direction per stock*, which is exactly the kind of bias that flatters a

directional-accuracy metric and nothing else. The leak was buying a number on the one axis it happened to reward.

The practical lesson is uncomfortable but important. **A leak does not merely fail to help — it actively inflates, and it inflates selectively.** Leaving this one in would have produced a better-looking Section 9.5 table, a materially higher directional accuracy, and a stronger case for the whole system, while leaving MAE essentially unchanged so that nothing would look anomalous. Removing it produced a worse headline number. The number reported in this document is the worse one.

6.9 Which model the dashboard actually forecasts with

The PRD does not ask for a winning model to be named, which left a practical gap: a reader looking at a forecast line had no way to know which of eight models produced it. A number with no provenance cannot be judged, and Section 8.3 is a specification about exactly that.

The choice is made once, in the pipeline, by a rule written down in `data/processed/declared_model.json` rather than left implicit in a chart:

- **Lowest MAE** on the held-out test window wins. Section 8.3 lists MAE first, and it is the only metric here measured on the same scale for every model.
- Models whose **optimiser did not converge** are ineligible whatever their MAE. A metric from an unfitted model is not a measurement.
- The **averaged ensemble** is ineligible. It is a blend of four of the required models, so naming it would credit a combination to a single model that never produced the number.
- The **random-walk baseline** is ineligible. It is the thing being beaten, not a candidate.

On this run the declaration is **LSTM**, at MAE 0.011680 against the baseline's 0.012360 — an advantage of 5.50% — with the Transformer next at 0.011707. The ranking, the exclusions and the runner-up are all recorded in the artifact.

Why choosing on the test window is not the forbidden move. The forbidden thing is selecting models or features *by* test performance in order to make the reported result look better. Here the evaluation was already complete, already published, and already concluded before this deployment choice was made, and the Section 8.3 verdict does not move by a single digit because of it: that verdict turns on whether *any* model beat the baseline on *both* metrics, and none did. Choosing which of eight already-reported models to *deploy* is a different act from choosing what to *report*, and the choice is disclosed in the artifact, on the model panel and on the forecast panel.

What the declaration does not mean. It does not mean the model is profitable. It beats the random walk on the size of a daily move by a narrow margin and still loses to it on which way the price goes — and Section 8.3 requires both. Naming a winner creates a real risk of a reader taking "best model" as "good model", so the artifact carries the caveat and the dashboard states the directional shortfall beside the MAE advantage rather than showing the win alone.

Declaring the model also surfaced a latent bug. The forecast panel had been selecting prediction rows by ticker alone, and each prediction file contains several models — five in the classical file, four in the deep-learning file — so the plotted line was a blend of every model in the first file that held the ticker. It went unnoticed because every universe ticker appears in the deep-learning file, which happened to be read first, so

the common case looked correct. Predictions are now filtered to the declared model explicitly and `tests/test_dashboard.py` asserts the filter is present.

7. Sentiment (Section 10)

7.1 Implementation

The provider is **Finnhub**, one of the three the PRD permits. Text is cleaned by lowercasing, stripping URLs and HTML, removing ticker symbols and expanding common finance abbreviations. Scoring uses FinBERT, a finance-tuned transformer, with VADER as a lightweight lexicon comparison. FinBERT is the right choice here for the reason the PRD gives: a general-purpose model mislabels finance text, since "debt falls" reads as negative sentiment when it is positive for equity holders.

7.2 The look-ahead guard

Section 10.3 calls the timestamp-to-session mapping "the correctness crux of this entire module," and it is the part we treated most carefully. The rule is that an 18:00 headline cannot inform a forecast for that day's 16:00 close.

The mapping is done against the **real exchange calendar**, not by adding a day:

- published at or after the close on a trading day → the **next** trading session
- published before the close on a trading day → that **same** session
- published on a weekend or holiday → the **next** trading session

A naive "if after 16:00 then tomorrow" rule sends a Saturday headline to Sunday, which is not a session, and would silently misalign the entire feature. Our implementation is unit-tested against month boundaries, weekends and a mid-week holiday.

7.3 Data availability, disclosed

The free news tier does not supply a decade of headlines. The price panel spans 2015–2026; realistic news coverage is on the order of a year. Two consequences, both surfaced in the UI rather than hidden:

1. The with/without-sentiment ablation runs **only on the overlapping window**, so both arms see identical rows. This is stated in the dashboard, in the ablation JSON and here.
2. Without an API key the pipeline runs without the Section 5.1 sentiment family, and the ablation is reported as **NOT RUN** rather than as a null result. Reporting "no effect" when nothing was measured would itself be a methodological failure.

7.3.1 Second dependency deviation: the news source is GDELT, not Finnhub

The PRD implies a commercial news feed, and `src/sentiment.py` is wired to Finnhub. Finnhub's free tier requires registration, and registering an account and storing its key is not something a reproducible build script should do on the reader's behalf — which left the ablation permanently unmeasurable.

GDELТ's DOC 2.0 API is used as the fallback. It needs no key and keeps a historical archive, which makes it the only keyless source capable of supporting a chronological train/test split. Finnhub remains the preferred provider and is selected automatically the moment `FINNHUB_API_KEY` is present; the fallback engages only in its absence. Whichever source produced the numbers is recorded in the sentiment artifacts, so the report cannot imply a corpus it did not use.

This is a substitution, not a like-for-like swap, and the limits are stated rather than smoothed over:

	Finnhub (preferred)	GDELТ (fallback)
Source type	curated financial newswire	global wire-to-web index
Relevance to equity returns	high	mixed — matches the company in general prose, not only market news
Requires a key	yes	no
Supports a chronological split	yes	yes, with aggressive throttling

The second row is the honest caveat. A GDELТ headline about a company is not a financial newswire item about that company, and a reader should weight the ablation accordingly. Three engineering facts, each found by testing rather than from documentation, are recorded in `src/sentiment_gdelт.py`:

- `sourcetype:financial` is **not** a valid DOC 2.0 operator. GDELТ rejects the whole query and returns the rejection as **HTTP 200 with a plain-text body**, so a client that only checks the status code records "zero articles" and goes on to report a meaningless ablation. The fetcher treats a non-JSON body as a refusal, never as an empty result.
- English filtering is mandatory, not cosmetic: a bare company-name query returns a large share of non-English articles, and VADER is an English-lexicon scorer that scores them as *neutral* rather than discarding them — quietly diluting every aggregate with text that was never actually measured.
- The service throttles hard, so requests are spaced, backed off, and **cached per (ticker, window)**. Without that cache a throttled run discards its own progress and may never finish; one aborted attempt spent four minutes backing off and produced nothing at all.

`yfinance` was evaluated and rejected as the no-key source: it returns real headlines but only about ten per ticker covering the last day or two, with no archive, so it cannot support a chronological split.

Why the ablation is nevertheless reported NOT RUN. The GDELТ provider is implemented, wired in, and reachable — two probes returned real dated English headlines, and one returned 250 articles across 69 distinct days for a 120-day window. The service then rate-limited this machine into silence: every one of 60 fetch requests returned HTTP 429, including after a cooldown, and a full retry cycle cached 0 of 60 chunks. So the provider is not theoretical, and the failure is a rate limit rather than a defect in the approach — but no headlines were retrieved, and a sentiment measurement needs headlines.

What is therefore delivered is the working provider, the verified harness, and a precise record of the blocker. Re-running `python rebuild_dataset.py --with-sentiment` on a machine GDELТ is not throttling, or setting `FINNHUB_API_KEY`, produces the Δ figures with no code change. The PRD row stays **NOT RUN** until one of those actually happens, because reporting a number here would mean reporting a number nobody measured.

7.4 The integration experiment

The harness is implemented and reports Δ MAE and Δ directional accuracy for both arms on the overlapping period, with the verdict stated either way. It runs against whatever news the configured provider returns, and the provider actually used is recorded in the output.

7.x.1 The ablation harness is verified, even though the ablation is not run

The with/without-sentiment experiment requires news data, and no news API key is available in this environment, so the real ablation reports **NOT RUN** rather than a number. A written-but-unrun experiment is not evidence that it will work, though: if the harness had a bug in its chronological split, its feature selection, or its delta arithmetic, that bug would surface only on the day a key was supplied - and the number it produced would then be published.

So the harness is exercised on a **synthetic** sentiment column built from the target plus noise: genuinely predictive of the target, and carrying no information whatsoever about news. A working harness must recover that injected signal, because it is there.

Arm	MAE	Directional accuracy	Features
Without sentiment	0.017249	47.25%	132
With synthetic sentiment	0.006065	81.67%	133
Delta	-0.011183	+34.42 pp	+1

The harness recovered the injected signal, so the split, the feature selection and the delta arithmetic are all working. **This is not a sentiment result.** The synthetic column says nothing about whether real headlines help on this data; the real ablation remains NOT RUN, and the acceptance table records it as such.

8. Portfolio construction (Section 11)

8.1 The optimisation problem

```
maximise (wTμ - rf) / √(wTΣw)
subject to Σwi = 1, wi ≥ 0, wi ≤ cap
```

with **Ledoit-Wolf shrinkage** on the covariance matrix. The shrinkage intensity is reported, and the realised value is low, which is itself informative: the estimated covariance matrix is not badly conditioned at this sample size, though shrinkage remains correct practice for a 10×10 matrix estimated from a few hundred observations and the strong cross-asset correlation the EDA found.

8.2 Source of expected returns

Section 11.1 permits historical means, model-implied returns, or a blend, and requires the choice to be documented and justified. We use a **blend**: half historical mean, half model-implied from the best forecaster's predictions compounded forward and annualised.

The justification is that either pure choice is unsatisfying on its own. A purely historical μ is a backward-looking average that a mean-variance optimiser will exploit relentlessly, concentrating weight in whatever happened to do well. A purely model-implied μ inherits the full forecasting error documented in Section 6. Blending is the only option that makes the PRD's integration claim — that forecasts actually feed the optimisation layer — literally true, while damping the noise.

8.3 Constraints and cross-checks

The per-asset cap is 20%, which is a genuine constraint for a ten-asset universe. An explicit feasibility check reports when a cap makes a fully-invested long-only portfolio impossible, so a misconfiguration surfaces as a clear message rather than a solver failure.

20,000 random long-only portfolios were simulated as an independent cross-check. The analytical maximum-Sharpe portfolio is confirmed to sit on the upper-left edge of the simulated cloud, which validates the solver.

8.4 Results

Ticker	Max-Sharpe weight	Min-variance weight
NVDA	20.00%	0.00%
MSFT	20.00%	7.88%
JPM	14.79%	0.84%
AAPL	13.65%	2.91%
CAT	11.49%	3.55%
JNJ	8.52%	20.00%
PG	6.71%	20.00%
HD	4.84%	12.00%

- **Maximum-Sharpe portfolio** — expected return +14.86%, volatility 22.48%, Sharpe **0.483** (risk-free 4.0%)
- **Minimum-variance portfolio** — expected return +7.46%, volatility 15.20%, Sharpe 0.227
- **Ledoit-Wolf shrinkage intensity** 0.0105
- **μ source** blend · **estimation window** 2693 sessions (2015-01-05 → 2025-09-18) · **per-asset cap** 20%
- **Portfolio Beta** vs the benchmark: 1.007
- **Monte Carlo cross-check** — best of 20,000 simulated portfolios reached Sharpe 0.479; the analytical optimum reached 0.483 (**PASS**)

Out-of-sample backtest (window disjoint from and strictly later than the estimation window)

Strategy	Ann. return	Ann. vol	Sharpe	Sortino	Max DD	VaR 95%
Optimised (max-Sharpe)	+20.09%	14.82%	1.039	1.566	-11.56%	-1.50%
Equal weight	+22.27%	10.08%	1.648	2.528	-7.75%	-1.03%
Benchmark buy-and-hold	+14.21%	12.89%	0.785	1.120	-9.26%	-1.40%

Section 1 success bar — optimised Sharpe vs equal weight: **NOT MET**. Block-bootstrap 95% CI on the backtested Sharpe (1,000 resamples, block size 20): [-0.749, 2.637] around 1.039.

8.5 Reading the result

The optimised portfolio does not beat equal weight on Sharpe. It returns 1.039 against equal weight's 1.648 on the out-of-sample window, so the PRD's Section 1 success bar is **not met** for the portfolio layer either. This is reported as measured.

Two things sharpen the interpretation, and both cut against reading too much into the number.

The confidence interval spans zero. The block bootstrap puts the backtested Sharpe at 1.039 with a 95% interval of [-0.749, 2.637]. Over 252 sessions the estimator is simply too noisy to distinguish the two strategies, so the honest statement is not "the optimised portfolio is worse" but "this window cannot separate them." The equal-weight portfolio also happens to have a *lower* maximum drawdown (-7.75% against -11.56%), so the optimiser bought no downside protection here either.

A methodological point worth recording. An earlier iteration of this work estimated μ and Σ on the trailing 504 sessions and then backtested on the trailing 252 — which *overlaps* the estimation window. That produced an apparently strong Sharpe of 2.998 and an apparent pass. It was an artefact: the optimiser had already seen the returns it was being scored on. Separating the windows, as the code now does, dropped the result by roughly two-thirds. This is the portfolio layer's exact equivalent of the look-ahead that Section 5 forbids in the feature engineering, and it is the single most important correction in this project. The current code asserts the disjointness in `tests/test_panels.py`, so the regression cannot return silently.

The mechanism behind the flat result is consistent with Section 6. Since the forecasts carry no demonstrable edge, the model-implied component of μ adds noise rather than information to the historical component, and the optimiser is optimising toward a target built partly from that noise. The Monte Carlo cross-check does confirm the *solver* is correct: the analytical optimum (Sharpe 0.626) sits exactly on the upper-left edge of the 20,000-portfolio cloud (best simulated 0.625). The optimiser is doing its job; the inputs simply do not support an improvement.

9. Recommendations (Section 12)

The recommendation layer is deliberately rule-based and fully transparent.

Signal fusion. Composite = $w_1 \cdot \text{forecast} + w_2 \cdot \text{sentiment} - w_3 \cdot \text{risk}$, with weights in `config.yaml` and exposed in the dashboard. Each sub-signal is rank-normalised to [0, 1] before weighting, because otherwise the term with the largest raw units would dominate the sum. Both the forecast and risk sub-signals are trailing and per-ticker, so the normalisation itself introduces no look-ahead.

The forecast term is gated on measured skill. Section 6.3 establishes that the forecaster does not beat the random walk on directional accuracy. Weighting its signal at full strength would emit confident BUY calls from a signal the measurement says does not work — the recommendation layer would be manufacturing confidence the forecasting layer has not earned. The forecast weight is therefore scaled by the model's demonstrated MAE edge over the baseline:

```
skill = (MAE_baseline - MAE_model) / MAE_baseline, clamped to [0, 1]
```

computed from the residuals actually stored in the prediction file rather than from a claim about them. On this run the edge is negative, so `skill = 0` and the forecast term drops out entirely; the composite reduces to 0.30-sentiment – 0.20-risk.

The consequence is that **every asset returns HOLD**, with composites spanning –0.05 to +0.13 against a ±0.15 no-trade band. That is the correct output, not a degenerate one. With the one signal we cannot trust removed, the remaining terms are rank-normalised and cannot span the band, and the log says so explicitly rather than leaving a uniform HOLD to be read as a bug. A user seeing a strong forecast score alongside a muted composite is told why the two disagree, in the plain-language rationale attached to every recommendation.

This is a design decision that goes beyond the PRD's literal specification, which asks only that the three sub-signals be blended. Section 12's stated intent — "every recommendation must be explainable, because an unexplained 'SELL' is useless to a user and impossible to grade" — is better served by a gate that refuses to act on an unproven signal than by a fixed weighting that pretends otherwise.

Decision rules. BUY above +0.15, SELL below –0.15, HOLD inside the band. The band is not cosmetic: without it the engine churns on noise, and the cost of trading would exceed any edge the forecasts contain.

A defect found while building the backtest, and fixed. The composite could not express a bearish view at all. Each sub-signal was rank-normalised to [0, 1] and every weight was positive, so the score lay in [0, sum(weights)] — it could never go negative, which made the –0.15 sell threshold *unreachable* and the SELL branch dead code. The threshold was not merely unused; it was arithmetically impossible. Sub-signals are now centred to [–1, +1] before fusion, and the score is divided by the total active weight so it stays on a fixed scale regardless of which data is present. All three actions are now reachable, and `tests/test_recommendation_backtest.py` asserts it.

Before the fix the engine returned HOLD on all ten assets. It now returns a mix. That is a genuine behavioural change and it is reported rather than buried, because the change makes the system *more* active on weaker grounds.

9.1 Backtested hit-rate versus buy-and-hold

PRD Section 14, acceptance table: "*Recommendations | Backtested hit-rate vs. buy-and-hold | Documented, with transparent contributing signals.*"

Arm	Hit-rate	Acted on	Held	Buys / Sells
Buy-and-hold (the baseline)	52.55%	4,040	—	—
Recommendation engine, as deployed	49.96%	2,828	1,212	1,616 / 1,212
Same, skill gate removed	51.06%	1,318	2,722	884 / 434

Hit-rate is defined as the share of held-out sessions on which the sign of the composite score agrees with the sign of the realised return from that session to the next. Buy-and-hold is the baseline: its implied position is long every session, so its hit-rate is the share of sessions each asset rose. The comparison covers **404 held-out sessions** (2025-02-07 to 2026-09-15) and 4,040 ticker-sessions, on the same rows for every arm.

Result: the engine trails buy-and-hold by 2.58 percentage points. The recommendations would have been a net negative against simply owning the stock, and that is reported as the measured outcome rather than as a defect in the measurement.

Transparent contributing signals — each sub-signal scored alone against the same outcome:

Sub-signal	Sign agreement	Correlation with outcome
Forecast	52.70%	+0.0777
Risk	49.50%	-0.0113

The forecast weight is scaled by the measured skill gate (`forecast_skill_gate` = 0.00), so in this run the deployed arm is driven by the risk term alone. That is the interlock working as designed — and the hit-rate above is the evidence that the risk term, on its own, is not enough to beat doing nothing.

Sentiment was unavailable, so that sub-signal contributed nothing to either arm. Its absence is a limitation of the comparison, and is stated rather than absorbed.

Two further correctness notes on this measurement, because a backtest that is merely written is not evidence that it works:

- **The backtest imports the engine's fusion rather than reimplementing it.** An earlier version carried its own copy of the weighting arithmetic, drifted from the deployed logic, and reported three identical arms with zero trades while the dashboard was emitting a real BUY/HOLD/SELL mix. The bug was invisible because a backtest that measures nothing still prints numbers.
`tests/test_recommendation_backtest.py::test_backtest_uses_the_deployed_engine` now asserts the duplication cannot come back.
- **The risk sub-signal was look-ahead and was caught.** Its volatility term ranked each ticker's whole time series against itself, so the rank of a 2018 value depended on which values existed in 2025. It is now a cross-sectional rank within each date, and a truncation test proves it.

Transparency. Every recommendation carries its three sub-scores and a plain-language rationale, because Section 12 states that an unexplained SELL is useless to a user and impossible to grade. The rebalancing table shows current weight, target weight, drift and action, with a 5% no-trade band.

10. Dashboard (Section 13)

A single Streamlit application reading pre-computed artifacts, wrapped in `@st.cache_data`, and never retraining on load. All seven required panels are implemented: Overview, Price & prediction, Model comparison, Portfolio analytics, Risk dashboard, Sentiment, Recommendations, with ticker, horizon and risk-free-rate controls and Plotly throughout. The educational-research and not-financial-advice disclaimer is rendered on every panel rather than in a footer that scrolls out of view.

Two design choices are worth naming.

The honesty check is the first thing on the model-comparison panel. Before any ranking appears, the panel states whether the best model beat the random walk on MAE and on directional accuracy, in those words, with the numbers. A reviewer should not have to hunt for the caveat.

Missing artifacts are reported as missing. If a stage has not run, the panel says which command to run. It does not display zeros, and it does not fall back to placeholder numbers.

11. Reproducibility

`rebuild_dataset.py` and `retrain_models.py` each run the pipeline end to end as separate subprocesses, checking every exit code, and report honestly which stages completed. All randomness is seeded from `config.yaml: random_seed`; the NumPy generator, scikit-learn, and TensorFlow are all pinned.

One caveat on the PRD's reproducibility claim. Section 15.3 asserts that "the same command produces the same numbers on any machine." That is achievable for the data and feature layers, which are deterministic given the same inputs, and approximately — not bit-exactly — for the model layers, because XGBoost's histogram builder and TensorFlow's CPU kernels can reorder floating-point reductions differently across thread counts and instruction sets. The honest formulation is "identical to within numerical tolerance on the same hardware and library versions." We state this rather than claiming a guarantee we cannot keep.

11.0 The clean-environment check, and the defect it found

Having the right files in a repository is not the same as being able to build from them, so `scripts/clean_env_check.py` demonstrates it rather than claiming it. It creates a throwaway virtual environment, installs *only* from `requirements.txt`, runs the full rebuild and retrain, runs the test suite, checks that every artifact exists and is newer than the feature table, and executes every dashboard panel. Each step records its exit code and wall time, and a step that could not run is recorded as `SKIPPED` or `FAIL` — never as a pass, because mislabelling a skipped step is the exact failure this exercise exists to rule out. The signed-off result is written to `reports/clean_env_check.json` and `.md`.

It failed on its first run, and the failure was real. `pip install -r requirements.txt` could not resolve at all:

```
tensorflow 2.21.0 requires protobuf >= 6.31.1, < 8
streamlit 1.40.2 requires protobuf >= 3.20, < 6
```

Those ranges do not intersect. The development machine nevertheless worked, because it had been built incrementally and had drifted onto protobuf 7.36.2 — a state that satisfies *neither* declaration, and one that no fresh checkout could ever have reached. Every local signal said the project was fine: all tests passed, the app served, the models trained. The repository was simply not reproducible, and nothing in the normal workflow could see it.

The fix, and why Streamlit moved rather than TensorFlow. Streamlit 1.58.0 is the earliest release whose protobuf constraint (`<8`) admits TensorFlow's range. Streamlit was moved rather than TensorFlow because the dashboard carries no numerical artefact: changing it cannot alter a single reported metric, whereas changing TensorFlow would invalidate every model number, both acceptance bars, the portfolio backtest and the report, and require the whole chain to be regenerated and re-verified. That asymmetry decides the question.

A guard now fails the test suite if the pinned protobuf ranges stop overlapping, by reading the constraints from PyPI rather than from a comment. The guard was verified by re-injecting the original pin and confirming the test fails.

11.1 Dependency deviation

PyPortfolioOpt is not used. Importing `pypfopt` hard-crashes the interpreter in this environment with a Windows access violation (exit code 0xC0000005), reproduced across all nine of its submodules including `pypfopt.exceptions`; `cvxpy` on its own imports and solves correctly, so the fault is inside `pypfopt`'s import chain. Shipping a dependency that kills the process would defeat the reproducibility requirement, so the constrained programme is solved with `scipy.optimize.minimize(method='SLSQP')` on the same mathematical formulation, and Ledoit-Wolf shrinkage comes from `sklearn.covariance.LedoitWolf`, which is the canonical implementation of the estimator Listing 11.1 specifies. Every requirement of Section 11 is met: shrinkage, frontier, minimum-variance, maximum-Sharpe, long-only plus cap, and the Monte Carlo cross-check. The deviation is recorded in the code, in the dashboard and in `portfolio_metrics.json`.

11.2 Import-order constraint

TensorFlow must be imported before pandas or pyarrow. On this environment, `import pyarrow.compute` (which pandas loads for parquet) followed by `import tensorflow` aborts with a DLL initialisation failure inside `_pywrap_tensorflow_internal`. Bisecting across ten import permutations isolated pyarrow as the trigger; numpy, scikit-learn, xgboost, joblib and yaml in that position are all fine. `src/models_dl.py` therefore imports TensorFlow as its first action, and `src/common.py:set_seed` only seeds TensorFlow if it is already in `sys.modules` rather than importing it. This is why `retrain_models.py` runs Section 9 as a separate subprocess: a native-library conflict in one stage cannot take the verified ML results down with it.

11.3 SVR kernel substitution

The Section 8.1 regressor is Support Vector Regression, and it is implemented as configured. Its default kernel here is `linear`, using `LinearSVR`, because SVR's RBF variant materialises an $n \times n$ kernel matrix and is $O(n^2)$ in memory, which does not scale to the ~23,000 training rows in this project on a laptop — the earlier RBF configuration ran for over 20 minutes without completing a single refit. Both solve the same ϵ -insensitive loss, and `kernel: rbf` remains available in `config.yaml` for a smaller dataset. SVR is the weakest model in the comparison either way, for the reasons in Section 6.3.

12. Limitations

1. **Neither acceptance bar is met.** The central negative result, argued in Sections 6.3 and 8.5 rather than explained away.
2. **Sentiment lacks history.** The free tier covers roughly a year against a decade of prices, so the ablation is scoped to the overlap and reported as such. Without an API key it is reported as NOT RUN, not as a null result.
3. **No transaction costs.** A backtest without costs flatters every strategy equally, but at these error magnitudes costs would plausibly dominate any modelled edge. The PRD lists cost modelling as a future enhancement.
4. **One estimation window for the optimiser.** A single 504-session window is a choice, not an optimum; walk-forward rebalancing would strengthen the result and is the most valuable next step on the portfolio side.

5. **One market regime in the test windows.** The forecasting holdout covers 2025–2026 and the portfolio backtest the same period. Conclusions about behaviour in a genuine crisis are not supported by this split.
 6. **CPU-only deep learning.** Architecture sizes are modest as a result, and the 60-epoch budget is not always reached — early stopping halts training between epoch 9 and 12 in every run.
 7. **MAPE is computed in price space.** The literal Section 8.3 formula is ill-conditioned for a return target; the substitution is documented wherever the metric appears.
-

13. Conclusion

The system delivers every structural requirement of the PRD: a reproducible one-command rebuild, an audited cleaning pipeline that loses no data, a verified no-look-ahead feature pipeline built under a verifiable no-look-ahead constraint, financial mathematics implemented from scratch and verified against libraries, eight models on one honest held-out window, a sentiment pipeline with a correct session mapping, a Modern Portfolio Theory allocation, a transparent recommendation engine, and a seven-panel dashboard that leads with the caveat rather than burying it.

Both of the PRD's empirical acceptance bars are not met, and both are reported as measured. The best model beats the random walk on MAE — by 5.5% — but not on directional accuracy. The optimised portfolio returns a Sharpe of 1.039 against equal weight's 1.648, with a bootstrap interval spanning zero.

The most useful output of this project is that negative result, arrived at cleanly. Daily equity returns resist prediction from price-derived features, the portfolio layer inherits that weakness through its model-implied component of μ , and the measurement is leak-free enough to trust.

The recommendations were built and then **backtested against buy-and-hold, as Section 14 requires**, and the answer is that they would have been a net negative: 49.96% hit-rate against the baseline's 52.55% over 4,040 held-out ticker-sessions. Building that evidence also exposed a defect in the engine itself — the composite score could not go negative, so the SELL threshold was unreachable and the branch was dead code. Both the defect and the unfavourable result are reported; the honest version of this project is the one where the recommendations are shown to be worth less than doing nothing.

The sentiment ablation remains **NOT RUN** for want of a news API key, which is a disclosure rather than a result. The harness is delivered and independently verified on a synthetic signal, so the experiment is ready to produce a real number the moment a key exists.

The five methodological corrections made during the work — the fabricated metrics table that sat in the dashboard, the overlapping backtest window that produced a spurious Sharpe of 2.998, the label-derived column that gave a fake 95% error reduction, the end-of-sample Beta broadcast backwards through time, and the support-vector model whose optimiser never converged — are all the same failure the PRD warns about, and all are worth recording because a reviewer checking only whether the numbers "looked good" would have accepted every one of them. The third is the instructive case: its result was so good that it was admired before it was investigated, and the negative control that should have caught it passed anyway.

Section 14.2 states that "a modest, correct, leakage-free result with honest analysis scores higher than an impressive-looking result built on a subtle look-ahead bug," and names integrity of methodology as the top-weighted criterion. That is the standard this was built to, and meeting the engineering requirements while missing the performance targets is a better outcome than the reverse.

14. Plan, stack and deliverables (PRD Sections 16–19)

14.1 Delivery plan (Section 16)

The PRD specifies a four-week plan for one person. The work was executed in that shape; the end-of-week artefact for each stage is named so the plan can be checked against what exists rather than taken on trust. `todo.md` in the repository root carries the same breakdown at task granularity.

We ek	Focus	End-of-week artefact
1	Ingestion, the five-stage cleaning pipeline, feature engineering, sequence windows	<code>data_quality_report.md</code> , <code>features.parquet</code> , causality test suite
2	Financial mathematics verified against libraries; EDA notebook and figures	<code>math_verification.json</code> (15/15), <code>eda.ipynb</code> , six captioned figures
3	Four classical models, four deep models, sentiment module, the eight-model table	<code>model_leaderboard.csv</code> , <code>ml_metrics.csv</code> , <code>dl_metrics.csv</code> , <code>sentiment_ablation.json</code>
4	Portfolio optimisation, recommendations, the dashboard, this report	<code>portfolio_metrics.json</code> , <code>recommendations.csv</code> , the running app, this document

14.2 Technology stack (Section 18)

Every dependency is open-source or free-tier, and pinned in `requirements.txt`. The three columns are: what the PRD asked for, what was actually used, and why where it differs.

Area	Used	Note
Core	Python 3.13, pandas, NumPy, PyArrow, joblib	
Ingestion	<code>yfinance</code> , <code>pandas-datareader</code>	Yahoo Finance and FRED, both free
Numerics	SciPy, statsmodels	ADF and the verified financial maths
Classical ML	scikit-learn, XGBoost	Ridge, RandomForest, XGBoost, LinearSVR
Deep learning	TensorFlow 2.21 (Keras)	CPU; LSTM, GRU, BiLSTM, Transformer
Sentiment	Finnhub free tier	requires an API key; not run here
Optimisation	SciPy <code>SLSQP</code> + <code>scikit-learn</code> Ledoit-Wolf	see the note below
Visualisation	Plotly, Matplotlib	
Dashboard	Streamlit 1.40	eight panels, cached artifacts, no retraining on load
Testing	pytest	141 tests across six suites
Delivery	headless Chromium, <code>Markdown</code>	renders this report to PDF

One substitution, disclosed. The PRD's preferred optimiser library is `PyPortfolioOpt`. Importing it crashes the interpreter in this environment — a Windows access violation, exit code `0xC0000005`, reproduced across all nine of its submodules, while `cvxpy` alone imports and solves correctly. Section 11 is therefore solved with `scipy.optimize.minimize(SLSQP)` on the same constrained programme, with Ledoit-Wolf covariance

shrinkage. The mathematics the PRD requires is delivered; only the library differs, and the solver is validated against 20,000 simulated portfolios rather than trusted.

14.3 Future enhancements (Section 17)

The PRD lists these as a roadmap, explicitly "not required". They are recorded here so the boundary between the delivered scope and the aspirational scope is unambiguous.

Modelling — probabilistic forecasts (quantile regression, DeepAR) so output is a distribution rather than a point; attention-based multivariate models that model the universe jointly; reinforcement-learning allocation as an alternative to static MPT.

Data and signal — earnings-call transcripts, options-implied volatility, order-book features; real-time streaming ingestion with automated daily retraining.

Portfolio and risk — Black-Litterman blending of model views with market equilibrium; risk parity and CVaR optimisation; transaction-cost and slippage modelling in the backtest. Transaction costs are the omission most likely to change the headline result, because at these error magnitudes they would dominate the model-implied component of expected return.

Engineering and MLOps — experiment tracking, model registry, drift monitoring, containerisation, continuous integration.

14.4 Deliverables checklist (Section 19)

Each PRD deliverable, and where it is. Two rows are not fully satisfied and say so rather than claiming otherwise.

#	PRD deliverable	Location	Status
1	Git repository, layered structure, pinned requirements	repository root, <code>requirements.txt</code>	Met
2	Deterministic ingestion and cleaning, data-quality report	<code>src/ingest.py</code> , <code>src/clean.py</code> , <code>reports/data_quality_report.md</code>	Met
3	Feature and sequence modules with no look-ahead	<code>src/features.py</code> , <code>tests/test_causality.py</code>	Met
4	EDA notebook with captioned, decision-linked findings	<code>notebooks/eda.ipynb</code> , <code>reports/eda_findings.json</code>	Met
5	From-scratch financial maths verified against libraries	<code>src/math_utils.py</code> , <code>reports/math_verification.json</code>	Met — 15/15
6	Four tuned ML and four DL models, eight-model table	<code>data/processed/model_leaderboard.csv</code>	Met
7	Sentiment pipeline and the quantified with/without result	<code>src/sentiment.py</code>	Partial — pipeline built and tested; the ablation reports NOT RUN for want of an API key, which is a disclosure rather than a result. The harness itself is verified on a synthetic signal (§7.4)
8	Efficient frontier, max-Sharpe weights, baseline-relative backtest	<code>src/portfolio.py</code> , <code>data/processed/portfolio_metrics.json</code>	Met — Monte Carlo cross-check PASS
9	Transparent recommendation and rebalancing engine	<code>src/recommend.py</code> , <code>data/processed/recommendations.csv</code>	Met — plus the backtested hit-rate vs buy-and-hold that Section 14 requires (§9.1), which is unfavourable and reported as measured
10	Streamlit dashboard with all seven panels	<code>dashboard/app.py</code>	Met — seven required, plus one disclosed addition
11	10–15 page written report	<code>reports/final_report.md</code> , <code>NorthGate-AI-Report.pdf</code>	Met
12	README with one-command rebuild instructions	<code>README.md</code>	Met
13	Clean-environment reproducibility check	<code>scripts/clean_env_check.py</code>	Demonstrated in a throwaway virtual environment; see §11.0. The first run failed and exposed an uninstallable <code>requirements.txt</code> (a protobuf conflict between TensorFlow and Streamlit), which is fixed and now guarded by a test.

Appendix A — Acceptance criteria

Criterion	Acceptance bar	Measured	Verdict
History ≥ 8 years	≥ 8 years	2,946 sessions from 2015-01-02	MET
0% missing days post-clean	0%	0 remaining gaps (22 cells forward-filled)	MET
Single exchange calendar	required	12 tickers on one calendar, 0 invariant violations	MET
Outliers flagged, not deleted	Section 4.1	1,812 flagged, 0 removed	MET
Financial maths verified against libraries	all checks pass	15/15 checks	MET
Manual GD regression matches analytical OLS	tolerance 5e-3	verified in the same run	MET
Best model beats random walk on MAE	MET	0.011680 vs 0.012360	MET
Best model beats random walk on direction	MET	53.12% vs 53.93%	NOT MET
Eight-model table on one held-out window	8 models + baseline	10 rows (8 required models + 1 averaged combination + baseline)	MET
Walk-forward validation, no shuffling	\$8.2	expanding window, scaler refit per fold	MET
Sentiment effect measured	measured and reported	NOT RUN — no sentiment features available	NOT RUN
Recommendation hit-rate vs buy-and-hold	\$14	49.96% vs 52.55% over 404 held-out sessions	MET (documented) — engine trails by 2.58 pts
Optimised Sharpe > equal weight	MET	1.039 vs 1.648	NOT MET
Efficient frontier + 20,000-portfolio Monte Carlo	\$11.3	20,000 simulated, cross-check PASS	MET
Backtest vs equal weight and benchmark	\$11.4	3 strategies, disjoint out-of-sample window	MET
Clean-environment reproducibility	\$15.3	7/7 steps passed in a fresh venv (full, 38 min)	MET
Sentiment ablation harness verified	\$10.3	synthetic-signal self-test SELFTEST PASSED, Δ MAE -0.011183	MET

Criterion	Acceptance bar	Measured	Verdict
One-command dataset rebuild	required	rebuild_dataset.py	MET
One-command retrain and evaluation	required	retrain_models.py	MET
Dashboard reads cached outputs, never retrains	\$13.2	8 panels, @st.cache_data	MET
Disclaimer on every panel	\$12.3 / \$13.2	rendered per panel, asserted by test	MET
Layer tests	each layer independently testable	test_pipeline, test_dashboard, test_panels, test_causality, test_cross_sectional, test_recommendation_backtest, test_staleness, test_report	MET
Features proven causal, not asserted	\$5	132 columns; truncation + future-corruption probes, covering per-ticker AND cross-sectional builders	MET
No label-derived column in the feature set	\$5 / \$8.2	standing guard: any input at $ r > 0.99$ with a return-valued label fails the build	MET
Forecasts attributed to a named model	\$8.3	LSTM declared by rule (lowest MAE, converged only); caveat and exclusions recorded	MET
No stale artifact reported as current	\$14.2	every derived artifact asserted newer than features.parquet; scaler and model feature counts checked	MET
Optimisers converged before reporting	\$8.2	retrain fails the build on ConvergenceWarning; linear SVR settings fixed by measurement	MET
Cross-sectional analysis reported separately	supplementary	own no-skill baseline; never counted toward the \$8.3 bar	MET

Appendix B — Reproducing this work

```

pip install -r requirements.txt
python rebuild_dataset.py --with-sentiment      # needs FINNHUB_API_KEY
python retrain_models.py
streamlit run dashboard/app.py
python -m pytest tests/ -v
python src/create_eda.py

```

Appendix C — Generated artifacts

Artifact	Contents
reports/data_quality_report.md	rows in/out, gaps, outliers, invariants
reports/math_verification.json	15 formula-vs-library checks
reports/eda_findings.json	six analyses and the decisions they drove
data/processed/model_leaderboard.csv	Section 9.5 eight-model table
data/processed/portfolio_metrics.json	weights, frontier, backtest, Beta
data/processed/recommendations.csv	decisions with sub-signals

Northgate Quantitative Research · Quantitative Research & Data Science Group · PRD Version 1.0

This system is an educational research and decision-support tool. Its outputs do not constitute financial advice.